Part 1: The FHE Dream and the Integer Approach

Fully Homomorphic Encryption over the Integers

Nirav Bhattad

March 27, 2026

Outline: Part 1

1. Motivation: The Cloud Computing Conundrum
2. The Quest for the Simplest Secure Scheme
3. Formalizing the FHE Dream
4. The Trivial Solution & Compactness
5. The Paradigm Shift: From Lattices to Integers
6. References

The Cloud Computing Conundrum

The Status Quo:

- We have robust solutions for protecting data in transit (e.g., TLS/SSL).
- We have robust solutions for protecting data at rest (e.g., AES).

The Cloud Computing Conundrum

The Status Quo:

- We have robust solutions for protecting data in transit (e.g., TLS/SSL).
- We have robust solutions for protecting data at rest (e.g., AES).

The Fundamental Vulnerability:

- Traditionally, to perform any meaningful computation on encrypted data, it must first be decrypted.
- Outsourcing heavy computations to an untrusted cloud server requires handing over your secret key or your raw, unencrypted data.

The Cloud Computing Conundrum

The Status Quo:

- We have robust solutions for protecting data in transit (e.g., TLS/SSL).
- We have robust solutions for protecting data at rest (e.g., AES).

The Fundamental Vulnerability:

- Traditionally, to perform any meaningful computation on encrypted data, it must first be decrypted.
- Outsourcing heavy computations to an untrusted cloud server requires handing over your secret key or your raw, unencrypted data.

The FHE Goal: Allow an untrusted party to take encrypted inputs, perform arbitrary unbounded computations on them, and return an encrypted result. When decrypted, this result matches exactly what the data owner would have obtained by computing on the plaintext locally.

The Caesar Cipher Analogy

"What is the simplest encryption scheme for which one can hope to achieve security?"

The Caesar Cipher Analogy

"What is the simplest encryption scheme for which one can hope to achieve security?"

The Caesar Cipher:

- It is incredibly simple, and ironically, "somewhat homomorphic".
- If you shift 'A' by 3 and 'B' by 3, you can add those shifts together.
- **Flaw:** It is entirely insecure.

The Caesar Cipher Analogy

"What is the simplest encryption scheme for which one can hope to achieve security?"

The Caesar Cipher:

- It is incredibly simple, and ironically, "somewhat homomorphic".
- If you shift 'A' by 3 and 'B' by 3, you can add those shifts together.
- **Flaw:** It is entirely insecure.

Modern Public-Key Crypto (RSA / ElGamal):

- Highly secure, but relies on computationally complex modular exponentiation.
- **Flaw:** They only support one type of operation (multiplication).

The Caesar Cipher Analogy

"What is the simplest encryption scheme for which one can hope to achieve security?"

The Caesar Cipher:

- It is incredibly simple, and ironically, "somewhat homomorphic".
- If you shift 'A' by 3 and 'B' by 3, you can add those shifts together.
- **Flaw:** It is entirely insecure.

Modern Public-Key Crypto (RSA / ElGamal):

- Highly secure, but relies on computationally complex modular exponentiation.
- **Flaw:** They only support one type of operation (multiplication).

Intuition: To compute any arbitrary boolean circuit (and thus any function), we need an algebraically complete set of gates. We strictly need both **addition** and **multiplication**.

Formalizing Homomorphic Encryption

We must rigorously define what a homomorphic scheme entails mathematically. A homomorphic public key encryption scheme, denoted $\mathcal{E}$, consists of four probabilistic polynomial-time (PPT) algorithms:

Formalizing Homomorphic Encryption

We must rigorously define what a homomorphic scheme entails mathematically. A homomorphic public key encryption scheme, denoted $\mathcal{E}$, consists of four probabilistic polynomial-time (PPT) algorithms:

$$\mathcal{E} = (\text{KeyGen}, \text{Encrypt}, \text{Decrypt}, \text{Evaluate})$$

The first three algorithms operate as standard public-key infrastructure. The "magic" lies entirely in the Evaluate algorithm.

Formalizing Homomorphic Encryption

We must rigorously define what a homomorphic scheme entails mathematically. A homomorphic public key encryption scheme, denoted $\mathcal{E}$, consists of four probabilistic polynomial-time (PPT) algorithms:

$$\mathcal{E} = (\text{KeyGen}, \text{Encrypt}, \text{Decrypt}, \text{Evaluate})$$

The first three algorithms operate as standard public-key infrastructure. The "magic" lies entirely in the Evaluate algorithm.

The Evaluate Algorithm: Takes as input a public key pk, a boolean circuit C, and a tuple of ciphertexts $\vec{c} = \langle c_1, ..., c_t \rangle$ (which correspond to the t input bits of C), and outputs a new ciphertext c_{out}.

Definitions of Correctness and FHE

Definition (2.1: Correct Homomorphic Decryption)

The scheme $\mathcal{E}$ is correct for a given t-input circuit C if, for any key-pair $(sk, pk) \leftarrow \text{KeyGen}(\lambda)$, any plaintexts $m_1, ..., m_t \in \{0, 1\}$, and any ciphertexts $c_i \leftarrow \text{Encrypt}(pk, m_i)$, the following holds with overwhelming probability:

$$\text{Decrypt}(sk, \text{Evaluate}(pk, C, \vec{c})) = C(m_1, ..., m_t)$$

Definitions of Correctness and FHE

Definition (2.1: Correct Homomorphic Decryption)

The scheme $\mathcal{E}$ is correct for a given t-input circuit C if, for any key-pair $(sk, pk) \leftarrow \text{KeyGen}(\lambda)$, any plaintexts $m_1, ..., m_t \in \{0, 1\}$, and any ciphertexts $c_i \leftarrow \text{Encrypt}(pk, m_i)$, the following holds with overwhelming probability:

$$\text{Decrypt}(sk, \text{Evaluate}(pk, C, \vec{c})) = C(m_1, ..., m_t)$$

Definition (2.2: Fully Homomorphic Encryption)

The scheme $\mathcal{E}$ is homomorphic for a class $\mathcal{C}$ of circuits if it is correct for all circuits $C \in \mathcal{C}$. The scheme $\mathcal{E}$ is **fully homomorphic** if it is correct for *all* boolean circuits (i.e., circuits of arbitrary depth and size).

Addressing the "Trivial Solution"

Can we trivially satisfy the definition of FHE right now? Yes, but it results in an inherently useless system.

Addressing the "Trivial Solution"

Can we trivially satisfy the definition of FHE right now? Yes, but it results in an inherently useless system.

Proof of Failure by Construction:

- **Evaluate Step:** Take the boolean circuit C and simply concatenate it to the input ciphertexts $c_1, ..., c_t$. Zero actual computation occurs on the server.
- **Decrypt Step:** The data owner decrypts the raw inputs, reads the attached circuit C, and runs the computation locally.

Addressing the "Trivial Solution"

Can we trivially satisfy the definition of FHE right now? Yes, but it results in an inherently useless system.

Proof of Failure by Construction:

- **Evaluate Step:** Take the boolean circuit C and simply concatenate it to the input ciphertexts $c_1, ..., c_t$. Zero actual computation occurs on the server.
- **Decrypt Step:** The data owner decrypts the raw inputs, reads the attached circuit C, and runs the computation locally.

The Flaw: This entirely defeats the purpose of outsourcing computation! The untrusted server does nothing, and the data owner is forced to do all the heavy computational work.

To prevent this mathematical loophole, FHE requires a strict constraint: **Compactness**.

The Compactness Constraint

Definition (2.3: Compactness)

A homomorphic scheme $\mathcal{E}$ is compact if there exists a fixed polynomial bound $b(\lambda)$ such that for any generated key-pair, any circuit C, and any sequence of ciphertexts $\vec{c}$, the size of the ciphertext output by $\text{Evaluate}(pk, C, \vec{c})$ is strictly bounded by $b(\lambda)$ bits, independently of the size or depth of C.

The Compactness Constraint

Definition (2.3: Compactness)

A homomorphic scheme $\mathcal{E}$ is compact if there exists a fixed polynomial bound $b(\lambda)$ such that for any generated key-pair, any circuit C, and any sequence of ciphertexts $\vec{c}$, the size of the ciphertext output by $\text{Evaluate}(pk, C, \vec{c})$ is strictly bounded by $b(\lambda)$ bits, independently of the size or depth of C.

Intuition: The crucial element here is that the size of the output ciphertext c_{out} must be bounded *independently* of the size or depth of the circuit C. You cannot just endlessly append data; the ciphertext must remain a fixed, compact size regardless of how massive the outsourced computation is.

The Paradigm Shift

The Breakthrough (2009): For 30 years, FHE was just a theoretical dream. Craig Gentry proved it was possible using complex algebraic geometry—specifically, ideal lattices over polynomial rings. Understanding it required heavy machinery from algebraic number theory.

The Paradigm Shift

The Breakthrough (2009): For 30 years, FHE was just a theoretical dream. Craig Gentry proved it was possible using complex algebraic geometry—specifically, ideal lattices over polynomial rings. Understanding it required heavy machinery from algebraic number theory.

The Integer Approach (2010): A year later, van Dijk, Gentry, Halevi, and Vaikuntanathan (DGHV) asked: *Did Gentry need all that heavy machinery?* The answer was no.

The Paradigm Shift

The Breakthrough (2009): For 30 years, FHE was just a theoretical dream. Craig Gentry proved it was possible using complex algebraic geometry—specifically, ideal lattices over polynomial rings. Understanding it required heavy machinery from algebraic number theory.

The Integer Approach (2010): A year later, van Dijk, Gentry, Halevi, and Vaikuntanathan (DGHV) asked: *Did Gentry need all that heavy machinery?* The answer was no.

Conceptual Simplicity: The DGHV scheme achieves FHE by constructing a "somewhat homomorphic" encryption scheme that merely uses elementary addition and multiplication over the integers. They proved Gentry's blueprint can be instantiated without complex algebraic structures.

Intuition: The Approximate-GCD Problem

If we are not using complex lattices, what is the security of this integer scheme based on?

Intuition: The Approximate-GCD Problem

If we are not using complex lattices, what is the security of this integer scheme based on?

It relies on a fundamental, highly intuitive number-theoretic puzzle: the **Approximate-GCD problem**.

Intuition: The Approximate-GCD Problem

If we are not using complex lattices, what is the security of this integer scheme based on?

It relies on a fundamental, highly intuitive number-theoretic puzzle: the **Approximate-GCD problem**.

- **The Trivial Case:** If you are given a set of integers that are *exact* multiples of a secret prime p, finding p is trivial using the standard Euclidean algorithm.

Intuition: The Approximate-GCD Problem

If we are not using complex lattices, what is the security of this integer scheme based on?

It relies on a fundamental, highly intuitive number-theoretic puzzle: the **Approximate-GCD problem**.

- **The Trivial Case:** If you are given a set of integers that are *exact* multiples of a secret prime p, finding p is trivial using the standard Euclidean algorithm.
- **The Cryptographic Twist:** What if we add just a tiny bit of random noise to each multiple?

Intuition: The Approximate-GCD Problem

If we are not using complex lattices, what is the security of this integer scheme based on?

It relies on a fundamental, highly intuitive number-theoretic puzzle: the **Approximate-GCD problem**.

- **The Trivial Case:** If you are given a set of integers that are *exact* multiples of a secret prime p, finding p is trivial using the standard Euclidean algorithm.
- **The Cryptographic Twist:** What if we add just a tiny bit of random noise to each multiple?

Conclusion: By simply adding noise, the problem of finding the common divisor p becomes exponentially hard. This elegant and simple concept serves as the absolute bedrock for the entire DGHV Fully Homomorphic Encryption scheme.

References

- Marten van Dijk, Craig Gentry, Shai Halevi, and Vinod Vaikuntanathan. *Fully Homomorphic Encryption over the Integers*. Eurocrypt, 2010.
- Craig Gentry. *Fully Homomorphic Encryption Using Ideal Lattices*. STOC, 2009.
- Ronald L. Rivest, Len Adleman, and Michael L. Dertouzos. *On Data Banks and Privacy Homomorphisms*. Foundations of Secure Computation, 1978.

Part 2: The Somewhat Homomorphic Scheme (SHE)

Fully Homomorphic Encryption over the Integers

Nirav Bhattad

March 27, 2026

Outline: Part 2

1. The Symmetric Foundation
2. Transitioning to a Public-Key Scheme
3. Homomorphism and The Noise Bottleneck
4. References

The Symmetric Foundation: Parameters

Before tackling the public-key version, we build the foundation using a beautifully simple symmetric-key scheme.

Parameters (based on security parameter λ):

- η: The bit-length of the secret key.
- ρ: The bit-length of the noise.

The Symmetric Foundation: Parameters

Before tackling the public-key version, we build the foundation using a beautifully simple symmetric-key scheme.

Parameters (based on security parameter λ):

- η: The bit-length of the secret key.
- ρ: The bit-length of the noise.

Key Generation – KeyGen(λ):

- Choose a random odd integer p from the interval $[2^{\eta-1}, 2^{\eta})$.
- This integer p is our **secret key**.

The Symmetric Foundation: Encryption

Encryption – Encrypt(p, m)**:** To encrypt a plaintext bit $m \in \{0, 1\}$, we choose two random variables:

- q: A large random integer (the quotient).
- r: A small random integer (the noise), where $|r| < 2^{\rho}$.

The Symmetric Foundation: Encryption

Encryption – Encrypt(p, m)**:** To encrypt a plaintext bit $m \in \{0, 1\}$, we choose two random variables:

- q: A large random integer (the quotient).
- r: A small random integer (the noise), where $|r| < 2^{\rho}$.

We compute the ciphertext c as:

$$c = p \cdot q + 2r + m$$

The Symmetric Foundation: Encryption

Encryption – Encrypt(p, m)**:** To encrypt a plaintext bit $m \in \{0, 1\}$, we choose two random variables:

- q: A large random integer (the quotient).
- r: A small random integer (the noise), where $|r| < 2^{\rho}$.

We compute the ciphertext c as:

$$c = p \cdot q + 2r + m$$

Intuition: Think of $p \cdot q$ as a giant "anchor point" on a number line. To hide our message m, we step a tiny, random distance away from this anchor (the $2r$ term) and add our bit. To anyone without the secret key p, this just looks like a massive, random integer.

The Symmetric Foundation: Decryption

Decryption – Decrypt(p, c)**:** To recover the message m, we compute:

$$m' = (c \pmod{p}) \pmod{2}$$

The Symmetric Foundation: Decryption

Decryption – Decrypt(p, c): To recover the message m, we compute:

$$m' = (c \pmod{p}) \pmod{2}$$

Why does this work? Let's break down the mechanics:

1. Substitute c: $(p \cdot q + 2r + m \pmod{p})$
2. Because $p \cdot q$ is an exact multiple of p, it vanishes under modulo p.
3. We are left with $(2r + m \pmod{2})$.
4. Because $2r$ is inherently even, it vanishes under modulo 2, leaving exactly m.

The Symmetric Foundation: Decryption

Decryption – Decrypt(p, c): To recover the message m, we compute:

$$m' = (c \pmod{p}) \pmod 2$$

Why does this work? Let's break down the mechanics:

1. Substitute c: $(p \cdot q + 2r + m \pmod{p})$
2. Because $p \cdot q$ is an exact multiple of p, it vanishes under modulo p.
3. We are left with $(2r + m \pmod 2)$.
4. Because $2r$ is inherently even, it vanishes under modulo 2, leaving exactly m.

The Critical Condition: Step 2 *only* yields $2r + m$ if the absolute value of the noise is strictly less than half the modulus: $|2r + m| < p/2$. If the noise is too large, it "wraps around" p, and decryption fails entirely.

Symmetric Decryption: A Visualizable Example

Let's put numbers to the theory.

Setup:

- Secret Key $p = 31$
- Message bit $m = 1$

Symmetric Decryption: A Visualizable Example

Let's put numbers to the theory.

Setup:

- Secret Key $p = 31$
- Message bit $m = 1$

Encrypt: Choose a random $q = 5$ and a small noise $r = 2$.

$$c = (31 \cdot 5) + 2(2) + 1 = 155 + 4 + 1 = 160$$

The ciphertext is 160.

Symmetric Decryption: A Visualizable Example

Let's put numbers to the theory.

Setup:

- Secret Key $p = 31$
- Message bit $m = 1$

Encrypt: Choose a random $q = 5$ and a small noise $r = 2$.

$$c = (31 \cdot 5) + 2(2) + 1 = 155 + 4 + 1 = 160$$

The ciphertext is 160.

Decrypt:

$$160 \pmod{31} = 5$$

$$5 \pmod{2} = 1$$

We successfully recovered $m = 1$. The noise $(2r + m = 5)$ was safely smaller than $p/2$ (15.5).

Public-Key Parameters & KeyGen

A true FHE scheme requires public-key infrastructure. We achieve this by publishing a massive list of "encryptions of zero."

Public-Key Parameters & KeyGen

A true FHE scheme requires public-key infrastructure. We achieve this by publishing a massive list of "encryptions of zero."

New Parameters:

- γ: The bit-length of the integers in the public key.
- τ: The number of integers in the public key.

Public-Key Parameters & KeyGen

A true FHE scheme requires public-key infrastructure. We achieve this by publishing a massive list of "encryptions of zero."

New Parameters:

- γ: The bit-length of the integers in the public key.
- τ: The number of integers in the public key.

Public-Key Generation – KeyGen(λ):

- The secret key remains the odd integer p.
- Generate $\tau + 1$ samples of the form $x_i = p \cdot q_i + 2r_i$.
- Ensure x_0 is the largest element, x_0 is odd, and its noise term r_0 is even.
- The public key is the tuple: $pk = \langle x_0, x_1, \ldots, x_\tau \rangle$.

Public-Key Encryption

Encryption – Encrypt(pk, m)**:** To encrypt a bit $m \in \{0, 1\}$:

- Choose a random subset $S \subseteq \{1, 2, \ldots, \tau\}$.
- Choose a small random noise integer r.

Public-Key Encryption

Encryption – Encrypt(pk, m)**:** To encrypt a bit $m \in \{0, 1\}$:

- Choose a random subset $S \subseteq \{1, 2, \ldots, \tau\}$.
- Choose a small random noise integer r.

Compute the ciphertext:

$$c = \left[m + 2r + 2 \sum_{i \in S} x_i \right]_{x_0}$$

Public-Key Encryption

Encryption – Encrypt(pk, m)**:** To encrypt a bit $m \in \{0, 1\}$:

- Choose a random subset $S \subseteq \{1, 2, \ldots, \tau\}$.
- Choose a small random noise integer r.

Compute the ciphertext:

$$c = \left[m + 2r + 2 \sum_{i \in S} x_i \right]_{x_0}$$

Intuition: Every x_i is effectively an encryption of 0. By summing a random subset of x_i, we are computing $0 + 0 + 0 \ldots$, creating a brand new, randomized encryption of zero. We then add our message m to it. The final modulo x_0 prevents the physical bit-size of the ciphertext from growing infinitely large.

Homomorphic Addition

Is this scheme homomorphic? Let's define Evaluate simply as integer addition and multiplication.

Given two ciphertexts: $c_1 = p \cdot q_1 + 2r_1 + m_1$ $c_2 = p \cdot q_2 + 2r_2 + m_2$

Homomorphic Addition

Is this scheme homomorphic? Let's define Evaluate simply as integer addition and multiplication.

Given two ciphertexts: $c_1 = p \cdot q_1 + 2r_1 + m_1$ $c_2 = p \cdot q_2 + 2r_2 + m_2$

Addition:

$$c_1 + c_2 = p(q_1 + q_2) + 2(r_1 + r_2) + (m_1 + m_2)$$

Homomorphic Addition

Is this scheme homomorphic? Let's define Evaluate simply as integer addition and multiplication.

Given two ciphertexts: $c_1 = p \cdot q_1 + 2r_1 + m_1$ $c_2 = p \cdot q_2 + 2r_2 + m_2$

Addition:

$$c_1 + c_2 = p(q_1 + q_2) + 2(r_1 + r_2) + (m_1 + m_2)$$

Observation: The structure is perfectly preserved! We have a new multiple of p, plus $2 \times$ (new noise), plus the sum of the messages. The noise grows **linearly** via the triangle inequality.

Homomorphic Multiplication

Now, let us multiply the ciphertexts:

$$c_1 \cdot c_2 = (p \cdot q_1 + 2r_1 + m_1)(p \cdot q_2 + 2r_2 + m_2)$$

Homomorphic Multiplication

Now, let us multiply the ciphertexts:

$$c_1 \cdot c_2 = (p \cdot q_1 + 2r_1 + m_1)(p \cdot q_2 + 2r_2 + m_2)$$

Expanding this yields:

$$c_1 \cdot c_2 = p\big(p \cdot q_1 q_2 + q_1(2r_2 + m_2) + q_2(2r_1 + m_1)\big)$$
$$+ 2\big(2r_1 r_2 + r_1 m_2 + r_2 m_1\big) + m_1 m_2$$

Homomorphic Multiplication

Now, let us multiply the ciphertexts:

$$c_1 \cdot c_2 = (p \cdot q_1 + 2r_1 + m_1)(p \cdot q_2 + 2r_2 + m_2)$$

Expanding this yields:

$$c_1 \cdot c_2 = p\big(p \cdot q_1 q_2 + q_1(2r_2 + m_2) + q_2(2r_1 + m_1)\big)$$
$$+ 2\big(2r_1 r_2 + r_1 m_2 + r_2 m_1\big) + m_1 m_2$$

Observation: The structure is still preserved. We have a massive multiple of p, plus $2 \times$ (mixed noise terms), plus the product of the messages $m_1 m_2$.

The Climax: The Noise Bottleneck

Look closely at the dominant noise term in the multiplication equation:

$$\mathbf{2r_1r_2}$$

The Climax: The Noise Bottleneck

Look closely at the dominant noise term in the multiplication equation:

$$\mathbf{2r_1r_2}$$

The Bottleneck: When we multiply two ciphertexts, the noise does not just add—it multiplies. It **squares** the magnitude of the noise.

The Climax: The Noise Bottleneck

Look closely at the dominant noise term in the multiplication equation:

$$\mathbf{2r_1r_2}$$

The Bottleneck: When we multiply two ciphertexts, the noise does not just add—it multiplies. It **squares** the magnitude of the noise.

The Consequence: Recall our strict decryption rule: the noise must remain strictly less than $p/2$. Because multiplication squares the noise, we can only perform a very limited number of sequential multiplications before the noise explodes past $p/2$, hopelessly destroying the underlying plaintext.

The Climax: The Noise Bottleneck

Look closely at the dominant noise term in the multiplication equation:

$$\mathbf{2r_1r_2}$$

The Bottleneck: When we multiply two ciphertexts, the noise does not just add—it multiplies. It **squares** the magnitude of the noise.

The Consequence: Recall our strict decryption rule: the noise must remain strictly less than $p/2$. Because multiplication squares the noise, we can only perform a very limited number of sequential multiplications before the noise explodes past $p/2$, hopelessly destroying the underlying plaintext.

This is why the scheme is currently only **Somewhat Homomorphic**. To achieve Fully Homomorphic Encryption, we must find a way to "clean" this noise.

References

É. Levieil and D. Naccache. *Cryptographic test correction*. Public Key Cryptography, Lecture Notes in Computer Science, 2008.

Part 3: Security and The Approximate-GCD Problem

Fully Homomorphic Encryption over the Integers

Nirav Bhattad

March 27, 2026

Outline: Part 3

1. The Security Reduction
2. Why Standard Algorithms Fail
3. Lattice Attacks and Parameter Justification
4. References

The Need for Rigorous Security

We have built a scheme that can homomorphically add and multiply. But is it actually secure?

The Need for Rigorous Security

We have built a scheme that can homomorphically add and multiply. But is it actually secure?

The Cryptographic Standard:

- In modern cryptography, we do not guess that a scheme is secure; we prove that breaking it is equivalent to solving a known hard mathematical problem.
- For RSA, the hard problem is integer factorization.
- For Gentry's original FHE, it is finding short vectors in ideal lattices.

The Need for Rigorous Security

We have built a scheme that can homomorphically add and multiply. But is it actually secure?

The Cryptographic Standard:

- In modern cryptography, we do not guess that a scheme is secure; we prove that breaking it is equivalent to solving a known hard mathematical problem.
- For RSA, the hard problem is integer factorization.
- For Gentry's original FHE, it is finding short vectors in ideal lattices.

Our Foundation: For our integer scheme, the authors reduce semantic security to a surprisingly simple-sounding problem: the **Approximate Integer Greatest Common Divisor** (Approximate-GCD).

Formalizing Approximate-GCD

Recall our noise distribution $\mathcal{D}_{\gamma,\rho}(p)$, which outputs integers of the form $x = pq + r$, where $x \approx 2^{\gamma}$, and $r \in (-2^{\rho}, 2^{\rho})$.

Formalizing Approximate-GCD

Recall our noise distribution $\mathcal{D}_{\gamma,\rho}(p)$, which outputs integers of the form $x = pq + r$, where $x \approx 2^\gamma$, and $r \in (-2^\rho, 2^\rho)$.

Definition (1.1: The (ρ, η, γ)-Approximate-GCD Problem)

Given polynomially many samples $x_0, x_1, \ldots, x_\tau$ drawn from $\mathcal{D}_{\gamma,\rho}(p)$ for a randomly chosen η-bit odd integer p, output the hidden integer p.

Formalizing Approximate-GCD

Recall our noise distribution $\mathcal{D}_{\gamma,\rho}(p)$, which outputs integers of the form $x = pq + r$, where $x \approx 2^{\gamma}$, and $r \in (-2^{\rho}, 2^{\rho})$.

Definition (1.1: The (ρ, η, γ)-Approximate-GCD Problem)

Given polynomially many samples $x_0, x_1, \ldots, x_{\tau}$ drawn from $\mathcal{D}_{\gamma,\rho}(p)$ for a randomly chosen η-bit odd integer p, output the hidden integer p.

The Search-to-Decision Reduction: The authors prove that if an attacker can predict the encrypted bit with a non-negligible advantage, they can be used as an oracle to extract the least-significant bit of the quotients. This allows the recovery of the secret key p via a modified binary GCD algorithm.
Conclusion: Semantic security holds as long as Approximate-GCD is hard.

The Naive Attack: The Euclidean Algorithm

Let's put on our attacker hats. We are given two public samples x_1 and x_2.

The Naive Attack: The Euclidean Algorithm

Let's put on our attacker hats. We are given two public samples x_1 and x_2.

The Trivial Case (No Noise): If $r = 0$, then x_1 and x_2 are exact multiples of p. We simply run the 2,000-year-old Euclidean algorithm, compute $\gcd(x_1, x_2)$, and immediately recover p.

The Naive Attack: The Euclidean Algorithm

Let's put on our attacker hats. We are given two public samples x_1 and x_2.

The Trivial Case (No Noise): If $r = 0$, then x_1 and x_2 are exact multiples of p. We simply run the 2,000-year-old Euclidean algorithm, compute $\gcd(x_1, x_2)$, and immediately recover p.

The Reality (With Noise): The noise ruins everything. The Euclidean algorithm relies on strict mathematical divisibility. With even a tiny bit of noise, the modulo reductions in the Euclidean algorithm cause the remainders to explode rather than shrinking neatly to zero.

The Sophisticated Attack: Continued Fractions

What about Continued Fractions? They are standard tools for finding rational approximations. Since $x_1/x_2 \approx q_1/q_2$, maybe q_1/q_2 appears in the continued fraction expansion of x_1/x_2?.

The Sophisticated Attack: Continued Fractions

What about Continued Fractions? They are standard tools for finding rational approximations. Since $x_1/x_2 \approx q_1/q_2$, maybe q_1/q_2 appears in the continued fraction expansion of x_1/x_2?.

The Mathematical Failure: Let's analyze the distance between the public ratio and the target quotient ratio:

$$\left| \frac{x_1}{x_2} - \frac{q_1}{q_2} \right| = \left| \frac{q_2 x_1 - q_1 x_2}{q_2 x_2} \right|$$

The Sophisticated Attack: Continued Fractions

What about Continued Fractions? They are standard tools for finding rational approximations. Since $x_1/x_2 \approx q_1/q_2$, maybe q_1/q_2 appears in the continued fraction expansion of x_1/x_2?.

The Mathematical Failure: Let's analyze the distance between the public ratio and the target quotient ratio:

$$\left|\frac{x_1}{x_2} - \frac{q_1}{q_2}\right| = \left|\frac{q_2 x_1 - q_1 x_2}{q_2 x_2}\right|$$

Substituting $x_i = q_i p + r_i$:

$$= \left|\frac{q_2(q_1 p + r_1) - q_1(q_2 p + r_2)}{q_2(q_2 p + r_2)}\right| \approx \left|\frac{q_2 r_1 - q_1 r_2}{p \cdot q_2^2}\right|$$

The Sophisticated Attack: Continued Fractions

What about Continued Fractions? They are standard tools for finding rational approximations. Since $x_1/x_2 \approx q_1/q_2$, maybe q_1/q_2 appears in the continued fraction expansion of x_1/x_2?.

The Mathematical Failure: Let's analyze the distance between the public ratio and the target quotient ratio:

$$\left|\frac{x_1}{x_2} - \frac{q_1}{q_2}\right| = \left|\frac{q_2 x_1 - q_1 x_2}{q_2 x_2}\right|$$

Substituting $x_i = q_i p + r_i$:

$$= \left|\frac{q_2(q_1 p + r_1) - q_1(q_2 p + r_2)}{q_2(q_2 p + r_2)}\right| \approx \left|\frac{q_2 r_1 - q_1 r_2}{p \cdot q_2^2}\right|$$

Intuition: The distance is dominated by the cross-noise term $(q_2 r_1 - q_1 r_2)/q_2^2$. This distance is simply *not small enough* to guarantee that the continued fraction algorithm will converge on q_1/q_2. The noise completely obscures the target.

The Geometric Threat: SDA and Lattices

Because simple algebraic attacks fail, we must turn to the geometry of numbers—specifically lattice reduction algorithms like LLL.

The Geometric Threat: SDA and Lattices

Because simple algebraic attacks fail, we must turn to the geometry of numbers—specifically lattice reduction algorithms like LLL.

Given many samples $x_0, x_1, \ldots, x_t$, the rational numbers $y_i = x_i/x_0$ form an instance of the **Simultaneous Diophantine Approximation (SDA)** problem.

The Geometric Threat: SDA and Lattices

Because simple algebraic attacks fail, we must turn to the geometry of numbers—specifically lattice reduction algorithms like LLL.

Given many samples $x_0, x_1, \ldots, x_t$, the rational numbers $y_i = x_i/x_0$ form an instance of the **Simultaneous Diophantine Approximation (SDA)** problem.
We apply Lagarias' SDA algorithm to the $(t+1)$-dimensional lattice L spanned by the rows of matrix M:

$$M = \begin{pmatrix} 2^\rho & 0 & 0 & \ldots & 0 \\ x_1 & -x_0 & 0 & \ldots & 0 \\ x_2 & 0 & -x_0 & \ldots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ x_t & 0 & 0 & \ldots & -x_0 \end{pmatrix}$$

Finding the Target Vector

The attacker wants to find the hidden quotients. Let's multiply M by the unknown vector of quotients $\vec{q} = \langle q_0, q_1, \ldots, q_t \rangle$:

$$\vec{v} = \langle q_0, q_1, \ldots, q_t \rangle \cdot M = \langle q_0 2^\rho, q_0 x_1 - q_1 x_0, \ldots, q_0 x_t - q_t x_0 \rangle$$

Finding the Target Vector

The attacker wants to find the hidden quotients. Let's multiply M by the unknown vector of quotients $\vec{q} = \langle q_0, q_1, \ldots, q_t \rangle$:

$$\vec{v} = \langle q_0, q_1, \ldots, q_t \rangle \cdot M = \langle q_0 2^\rho, q_0 x_1 - q_1 x_0, \ldots, q_0 x_t - q_t x_0 \rangle$$

Look closely at the components $q_0 x_i - q_i x_0$. Since $x_i = q_i p + r_i$, we can expand:

$$\begin{aligned} q_0 x_i - q_i x_0 &= q_0(q_i p + r_i) - q_i(q_0 p + r_0) \\ &= q_0 q_i p + q_0 r_i - q_i q_0 p - q_i r_0 \\ &= q_0 r_i - q_i r_0 \end{aligned}$$

Finding the Target Vector

The attacker wants to find the hidden quotients. Let's multiply M by the unknown vector of quotients $\vec{q} = \langle q_0, q_1, \ldots, q_t \rangle$:

$$\vec{v} = \langle q_0, q_1, \ldots, q_t \rangle \cdot M = \langle q_0 2^\rho, q_0 x_1 - q_1 x_0, \ldots, q_0 x_t - q_t x_0 \rangle$$

Look closely at the components $q_0 x_i - q_i x_0$. Since $x_i = q_i p + r_i$, we can expand:

$$\begin{aligned} q_0 x_i - q_i x_0 &= q_0(q_i p + r_i) - q_i(q_0 p + r_0) \\ &= q_0 q_i p + q_0 r_i - q_i q_0 p - q_i r_0 \\ &= q_0 r_i - q_i r_0 \end{aligned}$$

The Danger: The secret key p perfectly cancels out! This leaves $\vec{v}$ as a relatively short target vector in the lattice, with length roughly $2^{\gamma+\rho-\eta}\sqrt{t+1}$. If LLL finds $\vec{v}$, the attacker recovers the quotients, and subsequently recovers $p = \lfloor x_0/q_0 \rceil$.

The Climax: Defeating Lattice Reduction

Can LLL or BKZ find this target vector $\vec{v}$? This is where the paper's brilliant parameter selection saves the scheme.

The Climax: Defeating Lattice Reduction

Can LLL or BKZ find this target vector $\vec{v}$? This is where the paper's brilliant parameter selection saves the scheme.

Scenario 1: Attacker uses a small t (Small Lattice Dimension)

- Minkowski's bound dictates the presence of vectors bounded by the determinant.
- There are exponentially many *spurious* vectors in the lattice that are much shorter than our target $\vec{v}$. The target is hopelessly buried in geometric noise.

The Climax: Defeating Lattice Reduction

Can LLL or BKZ find this target vector $\vec{v}$? This is where the paper's brilliant parameter selection saves the scheme.

Scenario 1: Attacker uses a small t (Small Lattice Dimension)

- Minkowski's bound dictates the presence of vectors bounded by the determinant.
- There are exponentially many *spurious* vectors in the lattice that are much shorter than our target $\vec{v}$. The target is hopelessly buried in geometric noise.

Scenario 2: Attacker uses a massive t (Massive Lattice Dimension)

- To make $\vec{v}$ the absolute shortest vector, t must be huge.
- Known lattice reduction algorithms take time $\approx 2^{t/k}$ to find a 2^k approximation.
- Isolating $\vec{v}$ requires time roughly $2^{\gamma/\eta^2}$.

Summary of Secure Parameters

By setting the bit-length of the public key elements (γ) to be massively large specifically $\gamma = \omega(\eta^2 \log \lambda)$ the time $2^{\gamma/\eta^2}$ becomes super-polynomial. The attack is thwarted.

Final Secure Parameters:

- $\rho = \mathcal{O}(\lambda)$: Prevents brute-force guessing of the noise.
- $\eta = \tilde{\mathcal{O}}(\lambda^2)$: Supports the homomorphic noise budget (depth).
- $\gamma = \tilde{\mathcal{O}}(\lambda^5)$: Thwarts SDA and Lattice Reduction algorithms.

Summary of Secure Parameters

By setting the bit-length of the public key elements (γ) to be massively large specifically $\gamma = \omega(\eta^2 \log \lambda)$ the time $2^{\gamma/\eta^2}$ becomes super-polynomial. The attack is thwarted.

Final Secure Parameters:

- $\rho = \mathcal{O}(\lambda)$: Prevents brute-force guessing of the noise.
- $\eta = \tilde{\mathcal{O}}(\lambda^2)$: Supports the homomorphic noise budget (depth).
- $\gamma = \tilde{\mathcal{O}}(\lambda^5)$: Thwarts SDA and Lattice Reduction algorithms.

Trade-off: While this results in massive public keys (elements are $\tilde{\mathcal{O}}(\lambda^5)$ bits long), it guarantees rigorous security based solely on the hardness of the Approximate-GCD over the integers.

References

Marten van Dijk, Craig Gentry, Shai Halevi, and Vinod Vaikuntanathan. *Fully Homomorphic Encryption over the Integers*. Eurocrypt, 2010.

Part 4: Squashing the Decryption Circuit and Bootstrapping

Fully Homomorphic Encryption over the Integers

Nirav Bhattad

March 27, 2026

Outline: Part 4

1. The Bootstrapping Blueprint
2. The Depth Problem
3. Squashing the Decryption Circuit
4. The Shallow Decryption Circuit
5. References

The Crisis of the Noise Bottleneck

The Problem: Our Somewhat Homomorphic Encryption (SHE) scheme can add and multiply. However, every multiplication squares the noise.

The Crisis of the Noise Bottleneck

The Problem: Our Somewhat Homomorphic Encryption (SHE) scheme can add and multiply. However, every multiplication squares the noise.

Eventually, the noise exceeds the decryption boundary of $p/2$, and the ciphertext is permanently destroyed. How can we possibly evaluate a circuit of arbitrary depth?

The Crisis of the Noise Bottleneck

The Problem: Our Somewhat Homomorphic Encryption (SHE) scheme can add and multiply. However, every multiplication squares the noise.

Eventually, the noise exceeds the decryption boundary of $p/2$, and the ciphertext is permanently destroyed. How can we possibly evaluate a circuit of arbitrary depth?

The Solution: Bootstrapping (Gentry, 2009) If an encryption scheme is capable of homomorphically evaluating *its own decryption circuit*, plus one additional gate (like a NAND gate), it can evaluate *any* circuit of *any* depth.

How Bootstrapping Works

The Bootstrapping Mechanism:

1. Take a "noisy" ciphertext c (where noise is perilously close to $p/2$).
2. Encrypt c *again* under a new public key, resulting in a doubly-encrypted ciphertext.
3. Provide the server with an *encrypted* version of the secret key sk.
4. The server homomorphically evaluates the decryption algorithm on the inner ciphertext using the encrypted secret key.

How Bootstrapping Works

The Bootstrapping Mechanism:

1. Take a "noisy" ciphertext c (where noise is perilously close to $p/2$).
2. Encrypt c *again* under a new public key, resulting in a doubly-encrypted ciphertext.
3. Provide the server with an *encrypted* version of the secret key sk.
4. The server homomorphically evaluates the decryption algorithm on the inner ciphertext using the encrypted secret key.

Intuition: Because we evaluate the decryption circuit homomorphically, the output is not the raw plaintext. Instead, it is a **fresh ciphertext** encrypting the exact same message!
This new ciphertext has a fixed, smaller amount of noise determined solely by the depth of the decryption circuit. We have successfully "refreshed" the ciphertext.

Defining Bootstrappability

Definition (1.1: Bootstrappable Encryption)

Let $\mathcal{E}$ be a homomorphic encryption scheme, and let $\mathcal{C}_{\mathcal{E}}$ be the set of permitted circuits it can evaluate before noise exceeds the bounds.
Let $\mathcal{D}_{\mathcal{E}}$ be the set of augmented decryption circuits (decryption plus one additional gate).
We say $\mathcal{E}$ is **bootstrappable** if:

$$\mathcal{D}_{\mathcal{E}} \subseteq \mathcal{C}_{\mathcal{E}}$$

Defining Bootstrappability

Definition (1.1: Bootstrappable Encryption)

Let $\mathcal{E}$ be a homomorphic encryption scheme, and let $\mathcal{C}_{\mathcal{E}}$ be the set of permitted circuits it can evaluate before noise exceeds the bounds.
Let $\mathcal{D}_{\mathcal{E}}$ be the set of augmented decryption circuits (decryption plus one additional gate).
We say $\mathcal{E}$ is **bootstrappable** if:

$$\mathcal{D}_{\mathcal{E}} \subseteq \mathcal{C}_{\mathcal{E}}$$

In plain English: A scheme is bootstrappable if its noise budget is large enough to survive the homomorphic evaluation of its own decryption algorithm.

The Depth Problem: Integer Division

Can our SHE scheme handle its own decryption circuit? Let's look at the math.

The Standard Decryption Equation:

$$m' = (c \pmod{p}) \pmod{2}$$

The Depth Problem: Integer Division

Can our SHE scheme handle its own decryption circuit? Let's look at the math.

The Standard Decryption Equation:

$$m' = (c \pmod{p}) \pmod{2}$$

To evaluate this homomorphically as a boolean circuit, we must compute it over the integers:

$$c \pmod{p} = c - p\lfloor c/p \rceil$$

The Depth Problem: Integer Division

Can our SHE scheme handle its own decryption circuit? Let's look at the math.

The Standard Decryption Equation:

$$m' = (c \pmod{p}) \pmod{2}$$

To evaluate this homomorphically as a boolean circuit, we must compute it over the integers:

$$c \pmod{p} = c - p\lfloor c/p \rceil$$

The Roadblock: This requires computing $\lfloor c/p \rceil$, which relies on **integer division**. When translated into a boolean polynomial circuit, division has a massive multiplicative depth.

The Depth Problem: Integer Division

Can our SHE scheme handle its own decryption circuit? Let's look at the math.

The Standard Decryption Equation:

$$m' = (c \pmod{p}) \pmod{2}$$

To evaluate this homomorphically as a boolean circuit, we must compute it over the integers:

$$c \pmod{p} = c - p\lfloor c/p \rceil$$

The Roadblock: This requires computing $\lfloor c/p \rceil$, which relies on **integer division**. When translated into a boolean polynomial circuit, division has a massive multiplicative depth.

Our permitted circuits in the SHE scheme can only handle polynomials up to degree:

$$d \leq \frac{\eta - 4 - \log ||\hat{f}||_1}{\rho' + 2}$$

The standard division circuit's degree far exceeds this bound. We must **"squash"** the decryption circuit.

Squashing: Shifting the Burden

To make the scheme bootstrappable, we must reduce the multiplicative depth of the decryption circuit.

Squashing: Shifting the Burden

To make the scheme bootstrappable, we must reduce the multiplicative depth of the decryption circuit.

The Strategy: We shift the heavy computational lifting from the Decrypt algorithm (which is strictly constrained by the noise budget) to the Evaluate algorithm (which can be done in the clear by the server *before* the final homomorphic decryption step).

Squashing: Shifting the Burden

To make the scheme bootstrappable, we must reduce the multiplicative depth of the decryption circuit.

The Strategy: We shift the heavy computational lifting from the Decrypt algorithm (which is strictly constrained by the noise budget) to the Evaluate algorithm (which can be done in the clear by the server *before* the final homomorphic decryption step).

New Parameters:

- κ: Precision of rational numbers.
- Θ: Size of a public set of rational numbers.
- θ: The Hamming weight of a sparse secret subset.

Modifying the Key Generation

During KeyGen, alongside p and pk^*, we generate a "hint" about $1/p$.

Modifying the Key Generation

During KeyGen, alongside p and pk^*, we generate a "hint" about $1/p$.

1. The Sparse Secret Key: Choose a random Θ-bit vector $\vec{s} = \langle s_1, \ldots, s_\Theta \rangle$ with a very low Hamming weight θ. Let $S = \{i : s_i = 1\}$. This vector $\vec{s}$ becomes the new secret key.

Modifying the Key Generation

During KeyGen, alongside p and pk^*, we generate a "hint" about $1/p$.

1. The Sparse Secret Key: Choose a random Θ-bit vector $\vec{s} = \langle s_1, \ldots, s_\Theta \rangle$ with a very low Hamming weight θ. Let $S = \{i : s_i = 1\}$. This vector $\vec{s}$ becomes the new secret key.

2. The Public Rational Numbers: Generate a set $\vec{y} = \{y_1, \ldots, y_\Theta\}$ of rational numbers in $[0, 2)$ with κ bits of precision, such that the subset S sums to approximately $1/p$:

$$\sum_{i \in S} y_i \approx \frac{1}{p} \pmod 2$$

Specifically, $\left[\sum_{i \in S} y_i\right]_2 = \frac{1}{p} - \Delta_p$, where $|\Delta_p| < 2^{-\kappa}$.

Modifying the Key Generation

During KeyGen, alongside p and pk^*, we generate a "hint" about $1/p$.

1. The Sparse Secret Key: Choose a random Θ-bit vector $\vec{s} = \langle s_1, \ldots, s_\Theta \rangle$ with a very low Hamming weight θ. Let $S = \{i : s_i = 1\}$. This vector $\vec{s}$ becomes the new secret key.

2. The Public Rational Numbers: Generate a set $\vec{y} = \{y_1, \ldots, y_\Theta\}$ of rational numbers in $[0, 2)$ with κ bits of precision, such that the subset S sums to approximately $1/p$:

$$\sum_{i \in S} y_i \approx \frac{1}{p} \pmod 2$$

Specifically, $\left[\sum_{i \in S} y_i\right]_2 = \frac{1}{p} - \Delta_p$, where $|\Delta_p| < 2^{-\kappa}$.

Intuition: We hide a sparse subset sum within the public key. By giving the server these y_i fragments, the server can multiply the ciphertext by $1/p$ for us, saving the decryption circuit from doing the division!

Modifying Evaluate and Decrypt

When the server finishes evaluating a circuit, it holds a noisy ciphertext c^*. Before returning it, the server does post-processing in the clear.

Modifying Evaluate and Decrypt

When the server finishes evaluating a circuit, it holds a noisy ciphertext c^*. Before returning it, the server does post-processing in the clear.

Server Post-Processing: For $i \in \{1, \ldots, \Theta\}$, compute:

$$z_i = [c^* \cdot y_i]_2$$

keeping only $n = \lceil \log \theta \rceil + 3$ bits of precision. The server outputs c^* and $\vec{z} = \langle z_1, \ldots, z_\Theta \rangle$.

Modifying Evaluate and Decrypt

When the server finishes evaluating a circuit, it holds a noisy ciphertext c^*. Before returning it, the server does post-processing in the clear.

Server Post-Processing: For $i \in \{1, \ldots, \Theta\}$, compute:

$$z_i = [c^* \cdot y_i]_2$$

keeping only $n = \lceil \log \theta \rceil + 3$ bits of precision. The server outputs c^* and $\vec{z} = \langle z_1, \ldots, z_\Theta \rangle$.

Deriving the Squashed Decryption: We need to compute $c^*/p \pmod 2$.

$$\frac{c^*}{p} \approx c^* \sum_{i \in S} y_i \pmod 2$$

$$= \sum_{i=1}^{\Theta} s_i (c^* y_i) \pmod 2 \approx \sum_{i=1}^{\Theta} s_i z_i \pmod 2$$

Modifying Evaluate and Decrypt

When the server finishes evaluating a circuit, it holds a noisy ciphertext c^*. Before returning it, the server does post-processing in the clear.

Server Post-Processing: For $i \in \{1, \ldots, \Theta\}$, compute:

$$z_i = [c^* \cdot y_i]_2$$

keeping only $n = \lceil \log \theta \rceil + 3$ bits of precision. The server outputs c^* and $\vec{z} = \langle z_1, \ldots, z_\Theta \rangle$.

Deriving the Squashed Decryption: We need to compute $c^*/p \pmod 2$.

$$\frac{c^*}{p} \approx c^* \sum_{i \in S} y_i \pmod 2$$

$$= \sum_{i=1}^{\Theta} s_i (c^* y_i) \pmod 2 \approx \sum_{i=1}^{\Theta} s_i z_i \pmod 2$$

The New Decrypt Algorithm:

$$m' = \left[c^* - \left\lfloor \sum_{i=1}^{\Theta} s_i z_i \right\rceil \right]_2$$

The Climax: The Shallow Circuit

Look at our new decryption equation: $m' = \left[c^* - \lfloor \sum_{i=1}^{\Theta} s_i z_i \rceil \right]_2$.

Why is this brilliant?

- The server already gave us the rational numbers z_i.
- The secret key bits s_i are just 0 or 1.
- Our only job is to sum up numbers, and because $\vec{s}$ is sparse, only θ of them are non-zero!

The Climax: The Shallow Circuit

Look at our new decryption equation: $m' = \left[c^* - \lfloor \sum_{i=1}^{\Theta} s_i z_i \rceil\right]_2$.

Why is this brilliant?

- The server already gave us the rational numbers z_i.
- The secret key bits s_i are just 0 or 1.
- Our only job is to sum up numbers, and because $\vec{s}$ is sparse, only θ of them are non-zero!

Lemma (4.1: Elementary Symmetric Polynomials)

We use the "three-for-two" trick to reduce the sum. The i-th bit of the Hamming weight of a binary vector $\vec{\sigma}$ can be expressed as a polynomial of degree exactly 2^i using the 2^i-th elementary symmetric polynomial $e_{2^i}(\vec{\sigma})$.

The Climax: The Shallow Circuit

Look at our new decryption equation: $m' = \left[c^* - \lfloor \sum_{i=1}^{\Theta} s_i z_i \rceil\right]_2$.

Why is this brilliant?

- The server already gave us the rational numbers z_i.
- The secret key bits s_i are just 0 or 1.
- Our only job is to sum up numbers, and because $\vec{s}$ is sparse, only θ of them are non-zero!

Lemma (4.1: Elementary Symmetric Polynomials)

We use the "three-for-two" trick to reduce the sum. The i-th bit of the Hamming weight of a binary vector $\vec{\sigma}$ can be expressed as a polynomial of degree exactly 2^i using the 2^i-th elementary symmetric polynomial $e_{2^i}(\vec{\sigma})$.

Because we only need $\lceil \log \theta \rceil + 3$ bits of precision, the maximum degree required to compute the binary sum and rounding is bounded:

$$\text{Degree} \leq 64\theta \log^2 \theta$$

Bootstrapping Achieved

By setting $\theta = \lambda$, the degree of our squashed decryption polynomial is $\mathcal{O}(\lambda \log^2 \lambda)$.

Bootstrapping Achieved

By setting $\theta = \lambda$, the degree of our squashed decryption polynomial is $\mathcal{O}(\lambda \log^2 \lambda)$.

Theorem (4.2: Bootstrapping Achieved)

By setting the secret key length $\eta = \rho \cdot \Theta(\lambda \log^2 \lambda)$, the base SHE scheme can comfortably handle polynomials of degree $\mathcal{O}(\lambda \log^2 \lambda)$.
Because the degree of the squashed decryption circuit is strictly less than the maximum degree of the permitted polynomials, we have:

$$\mathcal{D}_{\mathcal{E}} \subset \mathcal{C}_{\mathcal{E}}$$

Therefore, the scheme is bootstrappable and, consequently, **Fully Homomorphic**.

Bootstrapping Achieved

By setting $\theta = \lambda$, the degree of our squashed decryption polynomial is $\mathcal{O}(\lambda \log^2 \lambda)$.

Theorem (4.2: Bootstrapping Achieved)

By setting the secret key length $\eta = \rho \cdot \Theta(\lambda \log^2 \lambda)$, the base SHE scheme can comfortably handle polynomials of degree $\mathcal{O}(\lambda \log^2 \lambda)$.
Because the degree of the squashed decryption circuit is strictly less than the maximum degree of the permitted polynomials, we have:

$$\mathcal{D}_{\mathcal{E}} \subset \mathcal{C}_{\mathcal{E}}$$

Therefore, the scheme is bootstrappable and, consequently, **Fully Homomorphic**.

Conclusion: By publishing an approximation of $1/p$ and forcing the evaluator to pre-compute partial products, we squashed decryption down to a simple sparse subset sum. We have achieved FHE using nothing but modular arithmetic over the integers.

References

- Marten van Dijk, Craig Gentry, Shai Halevi, and Vinod Vaikuntanathan. *Fully Homomorphic Encryption over the Integers*. Eurocrypt, 2010.
- Craig Gentry. *A Fully Homomorphic Encryption Scheme*. PhD Thesis, Stanford University, 2009.